Conflict-Based Coordinated Search for Multi-Agent Sensor Networks

Yusif Razzaq, *Instructor:* Eric Frew

***Abstract*—We study the problem of coordinating a team of sensing agents to localize a hidden target using information-seeking control strategies that reduce entropy in the belief distribution. We propose Conflict-Based Coordinated Search (CBCS), a hierarchical multi-agent planning framework that achieves near-optimal coordinated decision-making while maintaining computational tractability. CBCS integrates locally optimal planning with a centralized validation step, enabling agents to plan joint actions that minimize a given cost metric. We benchmark CBCS against several baseline strategies—including Greedy and Coordinated planners—under varying planning horizons and objective functions. Experimental results demonstrate that CBCS outperforms uncoordinated approaches for entropy reduction while incurring a significantly lower computational cost than fully coupled strategies. These findings highlight the utility of algorithmic innovations in reducing the computational burden associated with solving complex combinatorial problems.**

I. Introduction

The problem of coordinating a team of mobile sensing agents to efficiently infer a hidden state has broad applications in robotics, search and rescue, surveillance, and environmental monitoring. In these settings, autonomous agents must coordinate their actions to maximize information gain while operating under uncertainty and constrained computational resources. Global objectives often involve inter-agent dependencies, requiring solutions that reason jointly over the actions of all agents. However, as the size of the agent network grows, this coordination problem becomes increasingly complex due to the combinatorial nature of multi-agent decision-making.

A wide body of literature has addressed aspects of this problem through information-theoretic approaches. Hoffmann and Tomlin [1] developed a distributed control architecture that uses mutual information between sensors and particle filtering to coordinate mobile sensors for target localization. Similarly, Ryan and Hedrick [2] proposed a particle filter-based active sensing framework that uses receding-horizon control to maximize information gain. These information-driven formulations form the foundation of our entropy-informed control strategies, albeit with different approximation methods.

Bourgault et al. [3] approached the target search problem from a Bayesian perspective, developing control strategies that maximize the probability of detecting a target by maintaining a general-form probability density function (PDF). This formulation can accommodate arbitrary vehicle dynamics, sensor models, and target motion behaviors. However, their work focuses solely on single-agent scenarios, thereby sidestepping the complexities associated with multi-agent coordination.

Efforts toward multi-agent coordination have yielded strategies that attempt to balance scalability and performance. For instance, Zhou and Roumeliotis [4] explored optimal trajectory generation for multiple sensors using range-only measurements, proposing algorithms that relax computationally intractable formulations, thereby scaling linearly with the number of agents. Decentralized methods have also been proposed to manage coordination complexity. Chung et al. [5] introduced a decentralized gradient-descent strategy for dynamic target tracking, in which they formulated a control law that performs local minimizations with respect to a "quality of sensing" metric. Although efficient, such decentralized strategies suffer from sub-optimality due to approximations made when decoupling the coordination problem.

In contrast, the Multi-Agent Pathfinding (MAPF) [6] community has made significant strides in developing efficient algorithms to solve multi-agent coordination problems while providing optimality guarantees. Sharon et al. [7] introduced Conflict-Based Search (CBS), a two-level algorithm that resolves agent conflicts while preserving optimality. CBS performs a high-level search on a Conflict Tree, capturing conflicts between agents' plans and resolving them through fast, single-agent searches at the low level. This hierarchical approach enables CBS, in many cases, to explore fewer candidate plans compared to traditional approaches.

In this work, we introduce Conflict-Based Coordinated Search (CBCS), a hierarchical multi-agent planning framework that integrates information-theoretic control strategies with centralized coordination. CBCS combines locally optimal entropy-reducing plans with a global conflict resolution step that ensures feasibility and improves joint utility. By building on ideas from CBS and information gathering techniques, our approach delivers near-optimal agent coordination strategies, outperforming uncoordinated baselines while avoiding the combinatorial explosion associated with fully coupled planning.

The remainder of this paper is organized as follows: Section II formalizes the problem and environmental setup. Section III introduces key preliminaries, including the belief representation used to track the target state and the objective functions that guide control decisions. Section IV outlines the baseline approaches used for comparison, covering both greedy and coordinated strategies in one-step and receding-horizon settings. Section V presents the proposed CBCS framework, detailing its two-layer architecture for multi-agent coordination. Section VI analyzes the computational complexity of baseline methods and discusses key properties of the CBCS algorithm. Section VII reports simulation results, including a motivating example, benchmark evaluations under

varying environmental conditions, and an empirical study of scalability with respect to the number of agents and planning horizon. Finally, Section VIII concludes with a summary of findings and potential directions for future work.

II. Problem Formulation

A. Environment Description

We consider a team of N mobile agents $\mathcal{A} = \{a_1, a_2, \ldots, a_N\}$ operating within a bounded, discrete two-dimensional environment represented as a grid of size $W \times L$. The state space of the environment is denoted $\mathcal{X} \subset \mathbb{Z}^2$, where each cell corresponds to an integer-valued coordinate (x, y).

Each sensing agent a_i occupies a state $\mathbf{s}_k^i = (x_k^i, y_k^i) \in \mathcal{X}$ at discrete time step k. The agents can move according to a finite control set

$$\mathcal{U} = \{\texttt{up}, \texttt{right}, \texttt{down}, \texttt{left}, \texttt{stay}\} \subset \mathbb{Z}^2,$$

which corresponds to movement in the cardinal directions or remaining stationary. We define the mapping from control labels to displacement vectors as:

$$u = \begin{cases} (0,1) & \text{if } \texttt{up}, \\ (1,0) & \text{if } \texttt{right}, \\ (0,-1) & \text{if } \texttt{down}, \\ (-1,0) & \text{if } \texttt{left}, \\ (0,0) & \text{if } \texttt{stay}. \end{cases}$$

The agent dynamics follow a deterministic, discrete-time transition model $f : \mathcal{X} \times \mathcal{U} \to \mathcal{X}$:

$$f(\mathbf{s}_k, \mathbf{u}_k) = \mathbf{s}_{k+1} = \mathbf{s}_k + u_k, \quad \text{where } u_k \in \mathcal{U}. \tag{1}$$

Agent motions are constrained to remain within the grid bounds, i.e., transitions that would result in out-of-bounds positions are clipped to the nearest valid cell.

The goal of the team is to localize a stationary target located at an unknown position $\mathbf{x}_t = (x_t, y_t) \in \mathcal{X}$. We assume perfect communication among agents, allowing for centralized coordination, a shared belief representation, and joint planning.

III. Preliminaries

A. Belief Representation

The location of the target is modeled as an unknown variable with an associated probability distribution over the grid $\mathcal{X}$. At each time step k, the team maintains a belief distribution $b_k : \mathcal{X} \to [0, 1]$, where $b_k(\mathbf{x})$ represents the probability that the target is located at cell $\mathbf{x} = (i, j) \in \mathcal{X}$ at time k.

For compactness and computational efficiency, we represent the belief as a matrix $B_k \in [0, 1]^{W \times L}$, where the entry $[B_k]_{i,j}$ corresponds to the probability of the target being at grid cell (i, j). This matrix-based representation is equivalent to a discrete probability mass function over the finite grid.

Initially, the belief is uniform across all states:

$$[B_0]_{i,j} = \frac{1}{|\mathcal{X}|}, \quad \forall (i, j) \in \mathcal{X}, \tag{2}$$

reflecting maximal uncertainty regarding the target's location as shown in Figure 1a.

As the agents make noisy observations of the environment, their beliefs are updated recursively using Bayesian filtering. Let $\mathbf{z}_k = \{z_k^1, z_k^2, \ldots, z_k^N\}$ denote the collection of observations at time k. Then, the belief update takes the general form:

$$b_{k+1}(\mathbf{x}) = \frac{1}{\alpha} P(\mathbf{z}_k \mid \mathbf{x}) \cdot b_k(\mathbf{x}), \tag{3}$$

where $P(\mathbf{z}_k \mid \mathbf{x})$ is the likelihood of receiving observations $\mathbf{z}_k$ given the target is at location $\mathbf{x}$. The belief is normalized using the scalar α to ensure a valid probability distribution:

$$\alpha = \sum_{\mathbf{x} \in \mathcal{X}} P(\mathbf{z}_k \mid \mathbf{x}) \cdot b_k(\mathbf{x}). \tag{4}$$

This recursive process allows the agents to progressively refine their estimate of the target's location over time as new observations are gathered.

B. Observation Model

Each agent contributes a binary observation at every time step—either the agent detects the target (event D), or it does not (event $\neg D$). The probability of detection depends on the relative position between the sensing agent and the target. Specifically, we define the observation model as an exponential decay function of the Manhattan distance between the agent and the target.

Let $\mathbf{s} = (x_s, y_s) \in \mathcal{X}$ denote the sensor's location, and $\mathbf{x}_t = (x_t, y_t) \in \mathcal{X}$ denote a candidate target location. The probability that an agent located at $\mathbf{s}$ detects the target located at $\mathbf{x}_t$ is given by:

$$P(D \mid \mathbf{s}, \mathbf{x}_t) = 0.8 \cdot \exp\left(-\frac{d}{2}\right), \quad \text{where} \quad d = \|\mathbf{s} - \mathbf{x}_t\|_1, \tag{5}$$

and $\|\cdot\|_1$ denotes the Manhattan distance.

Because the observations are binary, the complete measurement likelihood model can be represented by the probability of detection and its complement:

$$P(z_k \mid \mathbf{x}) = \begin{cases} P(D \mid \mathbf{s}, \mathbf{x}) & \text{if } z_k = D, \\ 1 - P(D \mid \mathbf{s}, \mathbf{x}) & \text{if } z_k = \neg D. \end{cases} \tag{6}$$

Let $\mathbf{s}^i$ denote the position of agent i. Assuming conditional independence of observations given the target state, the total detection probability of observing $\mathbf{z}_k$ given target state $\mathbf{x}$ is:

$$P(\mathbf{z}_k \mid \mathbf{x}) = \prod_{i=1}^{N} P(z_k^i \mid \mathbf{s}_k^i, \mathbf{x}). \tag{7}$$

Figure 1b presents the total probability of detection, showing elevated likelihoods in regions near sensor locations.

To support efficient updates over the state space, we define a likelihood matrix generator $\mathcal{L} : \mathcal{X} \to [0, 1]^{W \times L}$, which returns the likelihoods of detecting the target in every grid cell from a given sensor position $\mathbf{s}$.

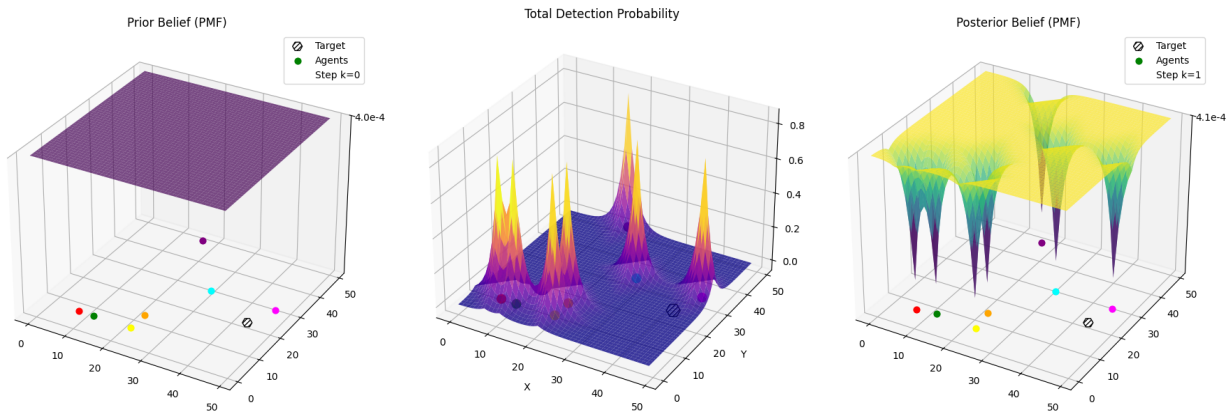


Fig. 1: Belief and detection model representations over a discretized grid. (a) Initial (prior) uniform belief at $k = 0$ (Eq. 2), (b) total detection probability over all agents (Eq. 7), and (c) posterior belief after N negative detections $\mathbf{z}_0 = \{\neg D\}^N$ (Eq. 9).

Formally, the matrix is defined as:

$$\mathcal{L}(\mathbf{s})[i,j] = P(D \mid \mathbf{s}, \mathbf{x}_t = (i,j)), \tag{8}$$

where $\mathbf{s} \in \mathcal{X}$ is fixed and $\mathbf{x}_t = (i,j) \in \mathcal{X}$ varies over all grid cells.

In practice, this matrix is precomputed for all sensor positions to enable fast likelihood evaluations during simulations. Then, to update a belief matrix B_k under a collection of observations $\mathbf{z}_k$, we recursively apply element-wise updates using the precomputed likelihood matrices:

$$B_{k+1} \propto \widetilde{B}_{k+1} = B_k \circ \prod_{i=1}^{N} \ell(z_k^i \mid \mathbf{s}_k^i), \tag{9}$$

where $\circ$ denotes the element-wise (Hadamard) product, and each likelihood term $\ell(z_k^i \mid \mathbf{s}_k^i)$ is defined as:

$$\ell(z_k^i \mid \mathbf{s}_k^i) = \begin{cases} \mathcal{L}(\mathbf{s}_k^i), & \text{if } z_k^i = D, \\ 1 - \mathcal{L}(\mathbf{s}_k^i), & \text{if } z_k^i = \neg D. \end{cases} \tag{10}$$

Finally, the updated belief is normalized by dividing the above expression by α.

$$\alpha = \sum_{i,j} \left[\widetilde{B}_{k+1}\right]_{i,j}, \tag{11}$$

ensuring that the sum of the updated distribution b_{k+1} over all states is equal to one. Figure 1c shows the result of updating the prior B_k with the likelihood of all sensors returning no detection (i.e., $\mathbf{z}_k = \{\neg D\}^N$).

C. Objective Functions

We investigate the use of different objective functions to determine which metric best guides the team's motion to localize the target. These metrics quantify the quality of future actions, given the agents' current positions and beliefs about the target location.

1) Maximize Detection Probability: We first consider a Bayesian approach that seeks to maximize the probability of detection, P_D, by computing the expected detection likelihood over the belief distribution. Specifically, the expected probability that a single sensor i at location $\mathbf{s}_{k+1}$ detects the target, given the current belief b_k, is given by:

$$\begin{aligned} P_D^i = P(D \mid \mathbf{s}_{k+1}^i, b_k) &= \sum_{\mathbf{x} \in \mathcal{X}} b_k(\mathbf{x}) \cdot P(D \mid \mathbf{s}_{k+1}^i, \mathbf{x}) \\ &= \sum_{m,n} \left[B_k \circ \mathcal{L}(\mathbf{s}_{k+1}^i)\right]_{m,n}, \end{aligned} \tag{12}$$

where $\mathcal{L}(\mathbf{s}_{k+1})$ is the likelihood matrix corresponding to the detection probability at each grid cell if the sensor were placed at $\mathbf{s}_{k+1}$.

This expression can be interpreted as the joint probability of the target being at location $\mathbf{x}$ and being detected by the sensor at $\mathbf{s}_{k+1}$, marginalized over all possible target locations:

$$P_D^i = \sum_{\mathbf{x} \in \mathcal{X}} P(\mathbf{x} = \mathbf{x}_t, D \mid \mathbf{s}_{k+1}). \tag{13}$$

To evaluate the expected detection performance of a team of sensors in a configuration $\mathcal{S}_{k+1} = \{\mathbf{s}_{k+1}^1, \ldots, \mathbf{s}_{k+1}^N\}$, we first compute the combined detection likelihood across all agents. Assuming conditional independence of observations, the aggregated likelihood matrix is given by:

$$\mathcal{L}(\mathcal{S}_{k+1}) = 1 - \prod_{i=1}^{N} \left(1 - \mathcal{L}(\mathbf{s}_{k+1}^i)\right), \tag{14}$$

where the product is taken element-wise across the matrices.

The corresponding expected detection probability over the joint sensor placements becomes:

$$\begin{aligned} P_D^J = P(D \mid \mathcal{S}_{k+1}, b_k) &= \sum_{\mathbf{x} \in \mathcal{X}} b_k(\mathbf{x}) \cdot P(D \mid \mathcal{S}_{k+1}, \mathbf{x}) \\ &= \sum_{i,j} \left[B_k \circ \mathcal{L}(\mathcal{S}_{k+1})\right]_{i,j} \end{aligned} \tag{15}$$

This objective captures the expected utility of placing sensors in configuration $\mathcal{S}_{k+1}$ naturally favoring placements that cover high-belief regions, thereby increasing the likelihood of detection and reducing uncertainty in the target state.

2) Maximize Mutual Information: We now consider an alternative objective that aims to maximize the *mutual information* (MI) gained through sensor placements. Mutual information quantifies the expected reduction in uncertainty (entropy) of the belief state after making an observation.

Formally, the mutual information is given by:

$$\text{MI} = H(b_k) - \mathbb{E}_z \left[H(b'_z) \right], \tag{16}$$

where $H(b_k)$ is the entropy of the prior belief, and b'_z is the posterior belief under observation $z \in \{D, \neg D\}$. Expanding the expectation over the two possible outcomes yields:

$$\text{MI} = H(b_k) - P_D \cdot H(b'_D) - (1 - P_D) \cdot H(b'_{\neg D}). \tag{17}$$

The posterior belief b'_z is computed via Bayes' rule:

$$b'_z(\mathbf{x}) = \frac{1}{\alpha} p(z \mid \mathbf{x}) \cdot b_k(\mathbf{x}), \quad \forall \mathbf{x} \in \mathcal{X}, \tag{18}$$

and the entropy over the discrete state space is defined as:

$$H(b) = -\sum_{\mathbf{x} \in \mathcal{X}} b(\mathbf{x}) \log b(\mathbf{x}). \tag{19}$$

In the case of greedy optimization, where each agent plans independently, the detection probability used in the mutual information calculation in (17) is agent-specific, $P_D = P_D^i$. In contrast, for coupled approaches, the mutual information uses the joint probability of detection across all agents $P_D = P_D^J$. In practice, we approximate the expectation over all possible joint measurements $E_{\mathbf{z}}[H(b'_{\mathbf{z}})]$ by considering only two scenarios: (i) at least one agent detects the target (with likelihood $\mathcal{L}(\mathcal{S}_{k+1})$), and (ii) no agents detect the target. This objective explicitly seeks to reduce entropy in the belief distribution, thereby encouraging sensor placements that explore previously uncertain or unexplored regions of the state space.

IV. Baseline Approaches

To investigate the performance of different multi-agent target-searching strategies, we establish a set of baseline algorithms that vary in both their degree of foresight (planning over a time horizon) and their method of coordinating agent actions (greedy vs coordinated planning). These algorithms are benchmarked and compared based on cumulative performance metrics such as entropy reduction, belief mass around the true target, and computational cost.

A. 1-Step Lookahead Strategies

We first examine a class of 1-step control strategies in which agents plan based on the immediate outcomes at time step $k+1$. We consider both a 1-StepGreedy planner, in which agent planning is decoupled, and a 1-StepCoordinated planner that reasons over the joint action space.

1) Greedy Action Planner: In this strategy, each agent selects its action sequentially, conditioned on the actions of all preceding agents. The decision is made by minimizing a cost function J, corresponding to the negation of mutual information gain or detection probability. Throughout, we use the term *greedy* to refer to this approach of sequential planning, and *1-step lookahead* to indicate the greedy-in-time nature of the decision-making process.

Algorithm 1 summarizes the procedure. The belief is initialized as a uniform distribution over the discretized state space; then, at each time step k, agents optimize their control inputs by minimizing local cost:

$$J_D^i = -P(D \mid \mathbf{s}_{k+1}^i, b_k^{i-1}) \tag{20}$$

or for mutual information-based planning:

$$J_{\text{MI}}^i = -\text{MI}(b' \mid \mathbf{s}_{k+1}^i, b_k^{i-1}). \tag{21}$$

Here, b_k^{i-1} is the prior belief updated based on the actions from agents 0 through $i-1$. When computing J_D^i, we assume that all preceding agents observed no detections ($\neg D$).

The single agent control law $c : \mathcal{X} \to \mathcal{U}$ can be expressed as:

$$c(\mathbf{s}_k^i) = \arg\min_{u \in \mathcal{U}} J(f(\mathbf{s}_k^i, u)). \tag{22}$$

After all agents select their control actions, the joint action $\mathcal{S}_{k+1} = \{\mathbf{s}_{k+1}^1, \ldots, \mathbf{s}_{k+1}^N\}$ is executed. Noisy binary measurements $z^i \in \{D, \neg D\}$ are taken at each sensor location and used to update the belief distribution via recursive Bayesian filtering as in (9).

Algorithm 1 1-StepGreedy

Initialize belief: $B_0 \leftarrow \text{Uniform}(|\mathcal{X}|)$
for each time step $k = 0$ to $T-1$ **do**
  Set working belief: $B_k^0 \leftarrow B_k$
  for each agent $i = 1$ to N **do**
    Select action: $u_k^i \leftarrow \arg\min_{u \in \mathcal{U}} J^i(\mathbf{s}_{k+1}^i, , B_k^{i-1})$
    Simulate motion: $\mathbf{s}_{k+1}^i \leftarrow f(\mathbf{s}_k^i, u_k^i)$
    Update working belief:

$$B_k^i \leftarrow \text{BayesUpdate}(B_k^{i-1}, \mathbf{s}_{k+1}^i, \neg D)$$

  end for
  Execute joint action: $\mathcal{S}_{k+1} \leftarrow \{f(\mathbf{s}_k^i, u_k^i)\}_{i=1}^N$
  Collect measurements: $\mathbf{z}_{k+1} \sim \{P(z \mid \mathbf{s}_{k+1}^i, \mathbf{x}_t)\}_{i=1}^N$
  Update: $B_{k+1} \leftarrow \text{BayesUpdate}(B_k, \mathcal{S}_{k+1}, \mathbf{z}_{k+1})$
end for

2) Coordinated Action Planner: In this strategy, agents collectively select a joint set of control inputs that minimizes a global cost objective. This approach contrasts with the 1-StepGreedy planner by reasoning over the full joint action space.

At each time step k, the planner evaluates the quality of all combinations of control actions and selects the joint action $\mathbf{u}_k = \{u_k^1, \ldots, u_k^N\} \in \mathcal{U}^N$ that minimizes either:

$$J_D = -P(D \mid \mathcal{S}_{k+1}, b_k) \quad \text{or,} \tag{23}$$

$$J_{\text{MI}} = -\text{MI}(b' \mid \mathcal{S}_{k+1}, b_k), \tag{24}$$

where $\mathcal{S}_{k+1} = \{f(\mathbf{s}_k^1, u_k^1), \ldots, f(\mathbf{s}_k^N, u_k^N)\}$ is the resulting joint state after applying the candidate action set.

The cooperative control law $\mathcal{C} : \mathcal{X}^N \to \Sigma$ can be expressed as:

$$\mathcal{C}(\mathcal{S}_k) = \arg\min_{\mathbf{u} \in \mathcal{U}^N} J(\mathcal{S}_{k+1}). \tag{25}$$

This strategy is summarized in Algorithm 2. Although more computationally intensive, the coordinated approach captures all possible configurations at $k+1$, enabling agents to plan optimally with respect to the global cost function.

Algorithm 2 1-STEPCOORDINATED

Initialize belief: $B_0 \leftarrow \text{Uniform}(|\mathcal{X}|)$
for each time step $k = 0$ to $T-1$ **do**
  Compute joint action:
  $$\mathbf{u}_k \leftarrow \arg\min_{\mathbf{u} \in \mathcal{U}^N} J\left(\{f(\mathbf{s}_k^i, u_k^i)\}_{i=1}^N\right)$$
  Apply joint control: $\mathcal{S}_{k+1} \leftarrow \{f(\mathbf{s}_k^i, u_k^i)\}_{i=1}^N$
  Collect measurements: $\mathbf{z}_{k+1} \sim \{P(z \mid \mathbf{s}_{k+1}^i, \mathbf{x}_t)\}_{i=1}^N$
  Update: $B_{k+1} \leftarrow \text{BayesUpdate}(B_k, \mathcal{S}_{k+1}, \mathbf{z}_{k+1})$
end for

B. Receding Horizon Strategies

We now introduce a class of *receding horizon strategies*, where agents plan a sequence of control actions over a finite time window of length $\mathcal{H}$, rather than selecting actions myopically at each step. These strategies retain the decision-making structures described in IV-A, but allow agents to reason about the impact of their actions across multiple future steps.

1) Greedy Action Planner: While still planning sequentially, an individual agent's actions are now coupled over $\mathcal{H}$-steps. The control sequence for agent i is denoted $\sigma_k^i = (u_k^i, \ldots, u_{k+\mathcal{H}-1}^i) \in \mathcal{U}^{\mathcal{H}}$, and the resulting planned states are $\mathbf{S}_{k+1}^i = (\mathbf{s}_{k+1}^i, \ldots, \mathbf{s}_{k+\mathcal{H}}^i)$, obtained through repeated application of the dynamics function f. For a detection-based objective, the cost function becomes:

$$J_D^i = -\sum_{h=1}^{\mathcal{H}} P(D \mid \mathbf{s}_{k+h}^i, b_{k+h-1}^{i-1}), \tag{26}$$

where b_{k+h-1}^{i-1} is the shared belief after accounting for all past actions from agents 1 to $i-1$ and prior steps of agent i until $k+h-1$. The receding horizon planner selects the first control u_k^i of the optimal sequence σ_k^i:

$$\sigma_k^i = \arg\min_{\sigma \in \mathcal{U}^{\mathcal{H}}} J_D^i(\vec{s}_i, b_k). \tag{27}$$

The belief is updated according to the full sequence σ_k^i so that downstream agents can plan effectively around the prescribed trajectory. This process is repeated for each agent, using a working copy of the belief state, which is updated in-place. Algorithm 3 outlines this procedure.

Algorithm 3 $\mathcal{H}$-STEPGREEDY

Initialize belief: $B_0 \leftarrow \text{Uniform}(|\mathcal{X}|)$
for each time step $k = 0$ to $T-1$ **do**
  $\mathbf{u}_k \leftarrow [\,]$, $\quad B_k^0 \leftarrow B_k$
  for agent $i = 1$ to N **do**
    $\sigma_k^i \leftarrow \arg\min_{\sigma \in \mathcal{U}^{\mathcal{H}}} J_D^i(\mathbf{S}_{k+1}^i, B_k^{i-1})$
    Append $\sigma_k^i[0]$ to $\mathbf{u}_k$
    Apply control sequence: $\mathbf{S}_{k+1}^i \leftarrow \{f(\mathbf{s}_k^i, u_k^i)\}_{i=1}^{\mathcal{H}}$
    Update $B_k^i \leftarrow \text{BayesUpdate}(B_k^{i-1}, \mathbf{S}_{k+1}^i, \{\neg D\}^{\mathcal{H}})$
  end for
  Apply joint control: $\mathcal{S}_{k+1} \leftarrow \{f(\mathbf{s}_k^i, u_k^i)\}_{i=1}^N$
  Collect measurements: $\mathbf{z}_{k+1} \sim \{P(z \mid \mathbf{s}_{k+1}^i, \mathbf{x}_t)\}_{i=1}^N$
  Update: $B_{k+1} \leftarrow \text{BayesUpdate}(B_k, \mathcal{S}_{k+1}, \mathbf{z}_{k+1})$
end for

2) Coordinated Action Planner: In contrast to sequential planning, the coordinated planner $\mathcal{H}$-STEPCOORDINATED evaluates the joint space of control sequences over the time horizon. The planner evaluates all possible combinations of joint sequences $\boldsymbol{\sigma}_k = (\sigma_k, \ldots, \sigma_{k+\mathcal{H}-1}) \in (\mathcal{U}^N)^{\mathcal{H}}$, selecting the one that maximizes the expected performance metric.

The detection-based objective becomes:

$$J_D = -\sum_{h=1}^{\mathcal{H}} P(D \mid \mathcal{S}_{k+h}, b_{k+h-1}), \tag{28}$$

where the joint detection probability accounts for the aggregate contribution of all agent positions at each future step, and b_{k+h-1} denotes the belief state after applying all previous steps of the joint control sequence. The planner selects the joint control sequence that minimizes J_D, and applies only the first action of each agent:

$$\boldsymbol{\sigma}_k = \arg\min_{\boldsymbol{\sigma} \in (\mathcal{U}^N)^{\mathcal{H}}} J_D(\mathcal{S}_{k+1}, \ldots, \mathcal{S}_{k+\mathcal{H}}, b_k), \tag{29}$$

$$\mathbf{u}_k = \boldsymbol{\sigma}_k[0] = \{u_k^1, \ldots, u_k^N\}. \tag{30}$$

The agents states' and global beliefs are updated through the dynamics and measurement models according to the planned joint action $\mathbf{u}_k$. Algorithm 4 summarizes the coordinated planning procedure.

Algorithm 4 $\mathcal{H}$-STEPCOORDINATED

Initialize belief: $B_0 \leftarrow \text{Uniform}(|\mathcal{X}|)$
for each time step $k = 0$ to $T-1$ **do**
  Compute best joint control sequence:
  $$\boldsymbol{\sigma}_k \leftarrow \arg\min_{\boldsymbol{\sigma} \in (\mathcal{U}^N)^{\mathcal{H}}} J_D\left(\{\mathcal{S}_{k+i}\}_{i=1}^{\mathcal{H}}, B_k\right)$$
  Extract first joint control: $\mathbf{u}_k \leftarrow \boldsymbol{\sigma}_k[0]$
  Apply joint control: $\mathcal{S}_{k+1} \leftarrow \{f(\mathbf{s}_k^i, u_k^i)\}_{i=1}^N$
  Collect measurements: $\mathbf{z}_{k+1} \sim \{P(z \mid \mathbf{s}_{k+1}^i, \mathbf{x}_t)\}_{i=1}^N$
  Update: $B_{k+1} \leftarrow \text{BayesUpdate}(B_k, \mathcal{S}_{k+1}, \mathbf{z}_{k+1})$
end for

V. Conflict-Based Coordinated Search

We now introduce Conflict-Based Coordination Search (CBCS), a novel approach to coordinated decision-making that extends the principles of Conflict-Based Search (CBS) beyond traditional collision-based planning. Instead of treating spatial collisions as conflicts, CBCS identifies sub-optimalities in joint plans—situations where an agent's local decision could be improved through better alignment with others—as the basis for defining conflicts. The algorithm operates over a Constraint Tree (CT), where each node represents a set of constraints applied to independent single-agent planning problems. In this work, we focus on 1-step lookahead scenarios, deferring extensions to receding horizon settings for future study.

A. Problem Formulation

To enable broader applicability of our method, we restate the multi-agent coordination problem in a general form that supports arbitrary discrete action spaces and cost metrics. Consider a set of agents $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ tasked with selecting actions that minimize a global cost function. Let $\mathcal{U}_i$ denote the discrete action space of agent a_i, and let $\mathbf{u} = (u_1, \dots, u_N) \in \mathcal{U}^N$ denote a joint action.

We define a global cost function $J : \mathcal{U}^N \to \mathbb{R}$ that quantifies the performance of a joint plan. With slight abuse of notation, we use $J(\mathbf{u})$ to denote the cost of the resulting joint state $\mathcal{S}_{k+1}$ after applying the joint action plan $\mathbf{u}$ to the current state $\mathcal{S}_k$, i.e.,

$$J(\mathbf{u}) = J\left(\left\{f(\mathbf{s}_k^i, u_k^i)\right\}_{i=1}^N\right),$$

where f denotes the single-agent transition function. The naive approach of evaluating all action combinations (as presented in Section IV-A2) incurs exponential time complexity $\mathcal{O}(|\mathcal{U}|^N)$, making it impractical for large teams or action spaces.

B. Conflict-Based Coordination Search (CBCS)

To address the combinatorial complexity of such problems, CBCS leverages the decoupled nature of agent plans in domains where global efficiency can be approximated with relatively few inter-agent conflicts. Inspired by CBS, it iteratively resolves conflicts between locally optimal agent plans by imposing constraints whenever their independent decisions are jointly suboptimal.

A *conflict* occurs between two agents (a_i, a_j) under a joint action $\mathbf{u} = (u_1, \dots, u_N)$ if there exists an alternative $u_j' \in \mathcal{U}_j$ such that:

$$J(u_1, \dots, u_j', \dots, u_N) < J(u_1, \dots, u_j, \dots, u_N).$$

In this case, a_j has an incentive to deviate from its original plan when given knowledge of a_i's intended action, implying suboptimal coordination.

Detailed in Algorithm 5, CBCS proceeds as follows:

1) Having each agent a_i independently select an optimal action $u_i \in \mathcal{U}_i$ given the current constraints.
2) Evaluating the resulting joint plan $\mathbf{u}$ under J.
3) If no pairwise conflict exists, returning $\mathbf{u}$ as a feasible coordinated plan.
4) If a conflict is detected between agents a_i and a_j, generating two child nodes:
   - One with a constraint forbidding u_i (i.e., $\mathcal{U}_i' = \mathcal{U}_i \setminus \{u_i\}$).
   - One with a constraint forbidding u_j (i.e., $\mathcal{U}_j' = \mathcal{U}_j \setminus \{u_j\}$).

   In each child, the unconstrained agents retain their actions while the constrained agent replans its action to minimize J.
5) Repeating the process until a conflict-free joint plan is found.

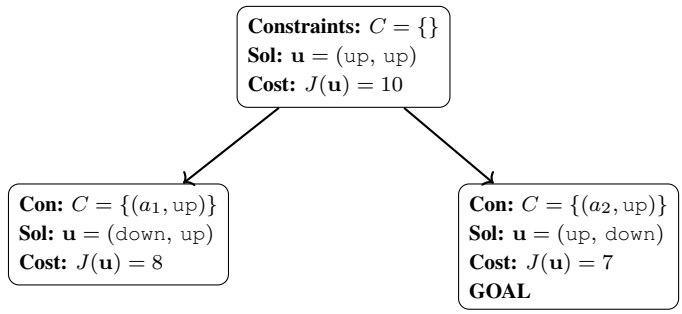


Fig. 2: An example of a Constraint Tree (CT).

Each node $\mathcal{N} = (\mathbf{u}, C, J(\mathbf{u}))$ in the CT maintains a candidate solution plan $\mathbf{u}$, the corresponding cost $J(\mathbf{u})$, and a constraint set $C = \{c, \dots\}$ where a constraint $c = (a_i, u)$ indicates agent a_i cannot take action u. Figure 2 illustrates a CT example: the root node begins with no constraints and assigns locally optimal actions to all agents. Upon detecting a coordination conflict in the original plan, two child nodes are generated, each enforcing a constraint on one of the conflicting agents. Nodes in the CT are expanded in increasing order of cost, systematically resolving conflicts by imposing more restrictions on agents' behavior at lower depths.

VI. Analysis

In this section, we provide a theoretical analysis of the algorithms presented in this paper.

A. Computational Complexity

We analyze the planning algorithms presented in Section IV in terms of their computational complexity, determining the number of actions evaluated by the planner at each time step.

- **Greedy Planning**: Each agent evaluates over its action space independently. For a planning horizon $\mathcal{H}$, each agent explores $|\mathcal{U}|^{\mathcal{H}}$ action sequences. The overall complexity is given by $\mathcal{O}(N \cdot |\mathcal{U}|^{\mathcal{H}})$.
- **Coordinated Planning**: Agents jointly evaluate action sequences, considering all possible $|\mathcal{U}|^N$ combinations of agents' actions over $\mathcal{H}$ steps. This leads to a combined action space of size $(|\mathcal{U}|^N)^{\mathcal{H}}$, and a time complexity of $\mathcal{O}(|\mathcal{U}|^{N \cdot \mathcal{H}})$.

Algorithm 5 Conflict-Based Coordination Search (CBCS)

Input: Action space $\mathcal{U}$, cost function $J : \mathcal{U}^N \to \mathbb{R}$
Output: Joint action $\mathbf{u}^* = (u_1^*, \ldots, u_N^*)$ minimizing J
$\mathbf{u} \leftarrow (\arg\min_{u_1 \in \mathcal{U}_1} J, \ldots, \arg\min_{u_N \in \mathcal{U}_N} J)$
Initialize OPEN $\leftarrow \{(\mathbf{u}, \emptyset, J(\mathbf{u}))\}$
while OPEN $\neq \emptyset$ **do**
  Pop $(\mathbf{u}, C, J(\mathbf{u}))$ with lowest cost from OPEN
  conflictFound $\leftarrow$ **False**
  for all $(a_i, a_j),\ i \neq j$ **do**
    for all $u_j' \in \mathcal{U}_j \setminus C_j$ **do**
      $\mathbf{u}' \leftarrow (u_1, \ldots, u_j', \ldots, u_N)$
      if $J(\mathbf{u}') < J(\mathbf{u})$ **then**
        conflictFound $\leftarrow$ **True**
        $C_1 \leftarrow C \cup \{(a_j, u_j)\}$
        $u_j^* \leftarrow \arg\min_{u \in \mathcal{U}_j \setminus C_1} J(u_1, \ldots, u_j, \ldots, u_N)$
        $\mathbf{u}_1 \leftarrow (u_1, \ldots, u_j^*, \ldots, u_N)$
        Add $(\mathbf{u}_1, C_1, J(\mathbf{u}_1))$ to OPEN
        $C_2 \leftarrow C \cup \{(a_i, u_i)\}$
        $u_i^* \leftarrow \arg\min_{u \in \mathcal{U}_i \setminus C_2} J(u_1, \ldots, u_i, \ldots, u_N)$
        $\mathbf{u}_2 \leftarrow (u_1, \ldots, u_i^*, \ldots, u_N)$
        Add $(\mathbf{u}_2, C_2, J(\mathbf{u}_2))$ to OPEN
        break
      end if
    end for
    if conflictFound then break
    end if
  end for
  if not **conflictFound then return** $\mathbf{u}$
  end if
end while

TABLE I: Time Complexity of Planning Algorithms.

Algorithm	Time Complexity
Random	$\mathcal{O}(N)$
1-Step Greedy	$\mathcal{O}(N \cdot \lvert\mathcal{U}\rvert)$
1-Step Coordinated	$\mathcal{O}(\lvert\mathcal{U}\rvert^N)$
$\mathcal{H}$-Step Greedy	$\mathcal{O}(N \cdot \lvert\mathcal{U}\rvert^{\mathcal{H}})$
$\mathcal{H}$-Step Coordinated	$\mathcal{O}(\lvert\mathcal{U}\rvert^{N\mathcal{H}})$

The time complexities of the baseline approaches are summarized in Table I. The exponential terms make coordinated and $\mathcal{H}$-step planning prohibitively expensive beyond small values of N and $\mathcal{H}$. As shown in our results in Section VII-G, this trade-off limits the practical scalability of joint planning strategies despite their potential performance advantages.

B. Properties of CBCS

CBCS mitigates the exponential complexity of joint planning by decomposing the global optimization problem into a series of constrained single-agent subproblems. While the joint action space $\mathcal{U}^N$ grows exponentially with the number of agents N, each local optimization in CBCS operates over the smaller action space $\mathcal{U}_i$ of a single agent, under a set of conflict constraints. Although the number of such subproblems can be exponential in the worst case (e.g., when many inter-agent conflicts are present), each subproblem can be solved efficiently. This makes CBCS particularly well-suited in domains where the agents are weakly coupled—such as large state space environments with limited sensor ranges—since an agent's greedy action is not likely to change based on information of a distant agent's plan. This effective independence requires considering relatively few constraints to produce high-quality joint actions.

We believe that certain structural properties must be imposed on the global cost function J in order for CBCS to provide optimality guarantees and/or bounds on its sub-optimality. For instance, if J is submodular with respect to individual agent contributions, then independent local optimizations are more likely to align with the global objective. Conversely, if J includes non-local effects—such as costs influenced by the geometry of multi-agent configurations—then CBCS may fail to detect conflicts, as no single-agent control change yields a lower-cost alternative, even though a coordinated set of changes could. It is also unclear whether conflicts must be symmetric for these properties to hold (i.e., whether a conflict between (a_i, a_j) necessarily implies a conflict between (a_j, a_i)). Further analysis of these properties is left to future work.

VII. Results

This section compares the performance of the baseline algorithms and CBCS under a variety of evaluation settings. The primary focus is on the relative effectiveness of each planner in achieving its task objectives. We compare algorithm performance across two objective functions—*information gain* and *target detection probability*—and explore how each scales with increasing agent counts and planning horizons.

A. Example Simulation

To illustrate the desired outcome of our multi-agent planning framework, we begin by presenting a representative example run. This simulation uses the 1-StepGreedy planner with the P_D optimization objective (20) in a 30×30 grid environment with three agents and a simulation time of 50 steps.

Figure 3 shows six key snapshots from the simulation. Each subplot depicts the environment at a different time step, as indicated in the upper-right corner of the plot. The heatmap represents the belief distribution (probability mass function), where yellow regions indicate higher belief of the target's location. The agent positions are indicated as circles, and the hexagon denotes the (fixed but unknown to the agents) target location.

Over time, the agents explore the environment by navigating to high-probability regions in the belief distribution. As the agents make observations, their beliefs become more concentrated, indicating greater certainty about the target's location. However, during intervals of missed detections (as in $k = [13,\ 22]$), entropy increases and belief around the target's location declines, as negative observations cause the belief to shift away from the true position.

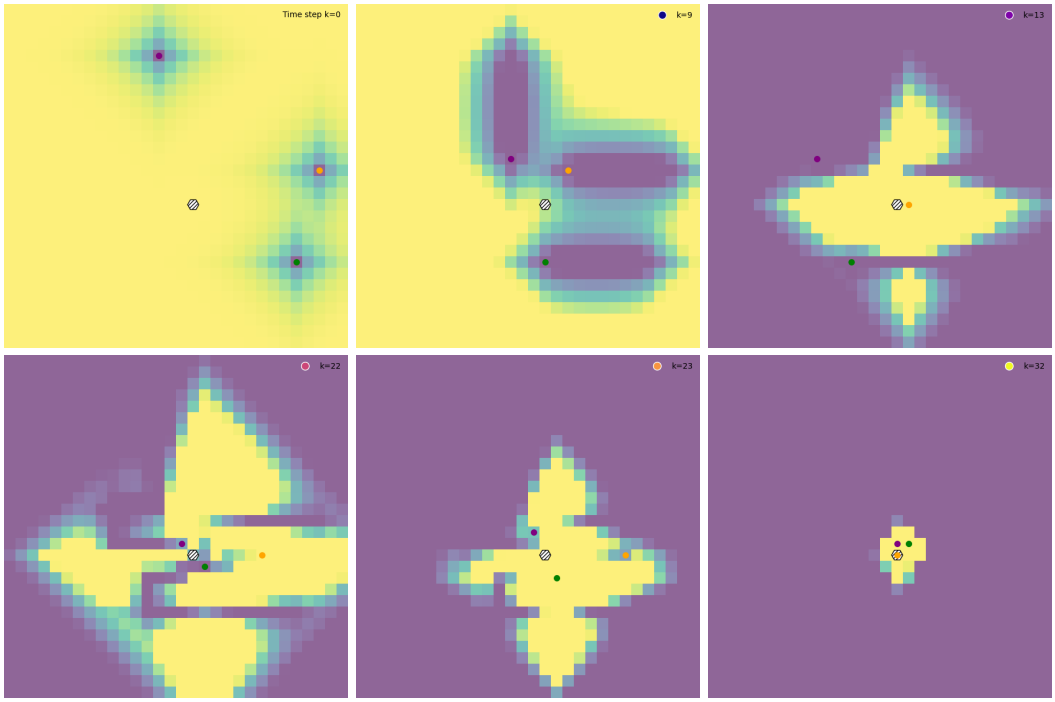


Fig. 3: Snapshots from a representative simulation run at increasing time steps. The PMF evolves as agents gather information and move toward high-likelihood regions (depicted in yellow). The white hexagon indicates the true target location.

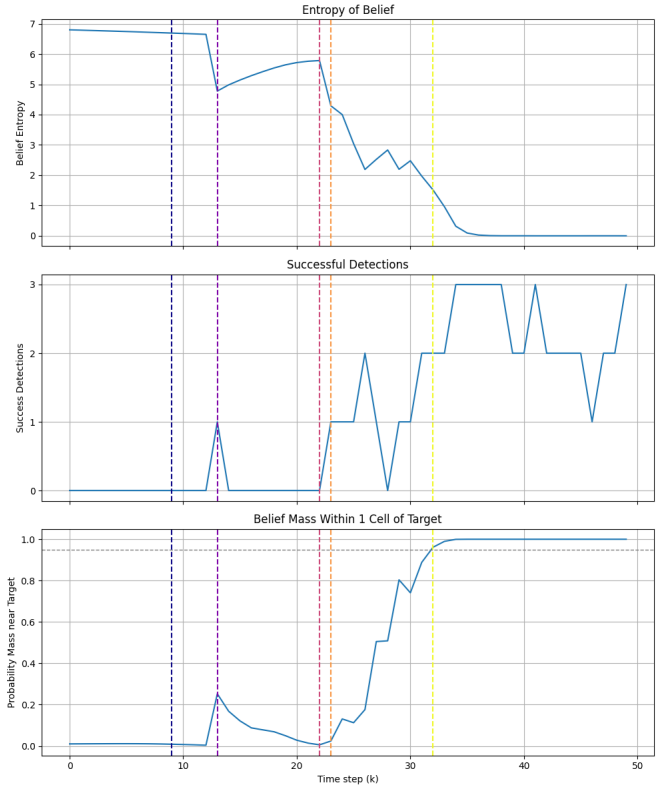


Fig. 4: Metrics for the example run: entropy (a), belief mass within 1-cell of the target (b), and number of detections (c). Vertical lines correspond to snapshots in Figure 3.

Figure 4 tracks the simulation's progress along various performance metrics. Subplot (a) shows the entropy of the belief distribution over time, demonstrating a reduction in uncertainty, particularly when detections are successful. Subplot (b) shows the total belief mass within a 1-cell neighborhood around the target, which increases as agents narrow in on the correct region. Subplot (c) indicates the number of successful detections at each time step, with a maximum of $N = 3$ possible detections. The vertical lines correspond to the snapshots presented in Figure 3 in chronological order.

B. Experimental Setup

To evaluate the performance of each planning strategy, we conducted multiple experiments across 100 randomized environments. All trials were performed in a 50×50 grid world, with agent and target positions randomly initialized at the start of each trial. To ensure a fair comparison, the same initial conditions were used for all planners within each trial.

For each trial, we record the following performance metrics:

- **Entropy Trajectory:** The evolution of belief entropy over time.
- **Probability Mass:** The probability mass in b_k concentrated near the true target location.
- **Detections:** The total number of successful target detections contributed by each agent.
- **Computation Time:** Average runtime per trial run.

Unless stated otherwise, all experiments use a default configuration of $N = 3$ agents and a planning horizon of $\mathcal{H} = 3$. This setting is used to maintain reasonable computation times, particularly for coordinated and receding-horizon planners.

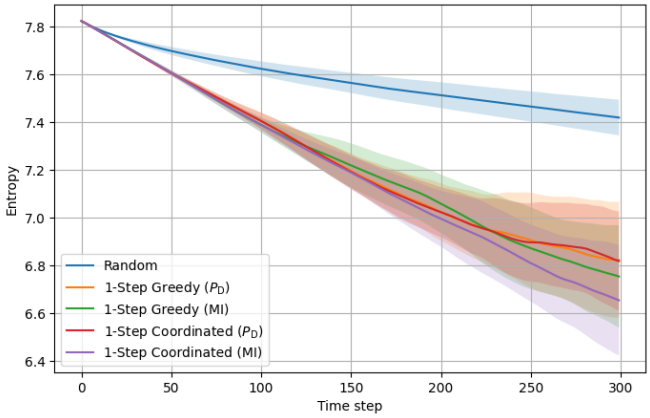
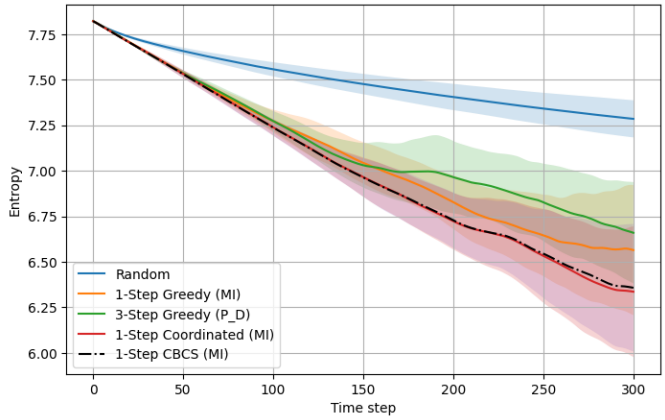
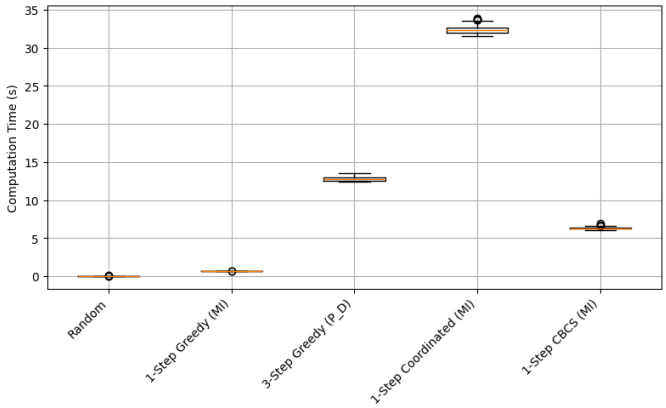


Fig. 5: Entropy trajectories when planning over an empty environment (no target exists).



(a) Entropy trajectories for different planning algorithms.



(b) Computation time per trail run for planning algorithms.

Fig. 6: Performance comparison of planning strategies with $N = 4$ agents. (a) Entropy reduction and (b) computation time per trial, demonstrating the efficacy of CBCS.

C. Information Gathering in an Empty World

We begin our evaluation by analyzing planner performance under two distinct optimization objectives: maximizing mutual information (MI) and maximizing detection probability. For this comparison, we focus on the 1-Step variants of the greedy and coordinated planners.

To isolate the impact of the objective function on entropy reduction, we disable detection events during this experiment, effectively removing the target from consideration. This eliminates abrupt entropy drops caused by successful detections and removes the randomness introduced by simulated measurements, enabling deterministic evaluation of planning strategies across randomized initial conditions.

We compare five planning approaches: RANDOM, 1-STEPGREEDY, 1-STEPCOORDINATED, $\mathcal{H}$-STEPGREEDY, and 1-STEPCBCS planning. Each algorithm was evaluated over 100 independent trials.

We evaluate five planner configurations: RANDOM, 1-STEPGREEDY with detection objective J_D^i (20), 1-STEPGREEDY with mutual information objective J_{MI}^i (21), 1-STEPCOORDINATED with detection objective J_D (23), and 1-STEPCOORDINATED with mutual information objective J_{MI} (24). The mean and $1 - \texttt{std}$ bounds of entropy trajectories over 100 trials are shown in Figure 5.

While all planners significantly outperform the RANDOM baseline (which selects a random control at each time step), mutual information-based planning consistently achieves greater entropy reduction. Notably, the 1-STEPGREEDY planner using the MI objective surpasses even the 1-STEPCOORDINATED planner using the detection objective, underscoring the advantage of MI-driven strategies when uncertainty reduction is the primary goal.

D. Performance of CBCS

We now evaluate the performance of CBCS against several baseline planners. For this experiment, we increase the number of agents to $N = 4$, disable detections, and consider only the mutual information (MI) objective for each planner: 1-STEPGREEDY, 1-STEPCOORDINATED, and 1-STEPCBCS. Additionally, we include a 3-STEPGREEDY planner which is evaluated using the J_D^i objective (26).

Figure 6 presents the results, where subplot (a) illustrates the entropy evolution over time, and subplot (b) shows a boxplot of per-trial runtimes. Notably, CBCS achieves near-optimal performance, closely tracking the fully coupled 1-STEPCOORDINATED planner in terms of entropy reduction. Despite this comparable performance, CBCS offers a significant computational advantage—achieving a 5× reduction in runtime compared to the coordinated planner.

This efficiency gain becomes increasingly pronounced as the number of agents grows, supporting the scalability advantages discussed in Section VII-G.

E. Target Localization

With detections enabled, we evaluate how incorporating the measurement model affects planner performance in localizing the target. In this experiment, we benchmark all baseline

planners from Section IV and CBCS. Due to its high computational cost, the $\mathcal{H}$-STEPCOORDINATED is excluded from this comparison.

Figure 7 summarizes the results across 100 randomized trials, showing the performance of each method with respect to three metrics: entropy reduction, belief mass around the true target location, and number of successful detections. Unlike the results in Subsection VII-C, here the detection-maximizing strategies performed as well as their mutual-information counterparts. Interestingly, the optimal MI-seeking coordinated planner and the near-optimal CBCS achieved comparable performance to other methods while relying on significantly fewer measurements (approximately 50% fewer on average). This suggests that a small number of high-quality (informative) observations can rival the effectiveness of numerous low-value ones. However, due to high variability across runs, definitive conclusions cannot be drawn.

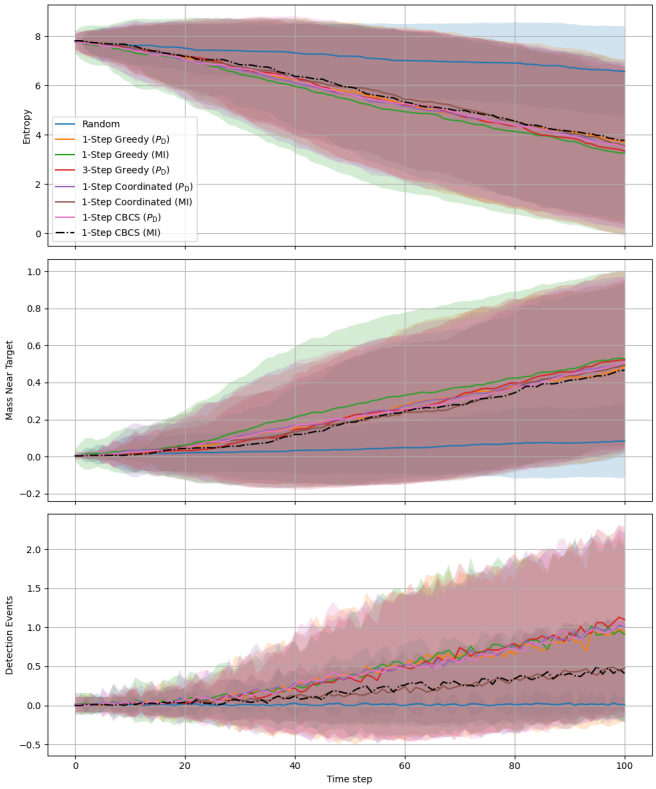
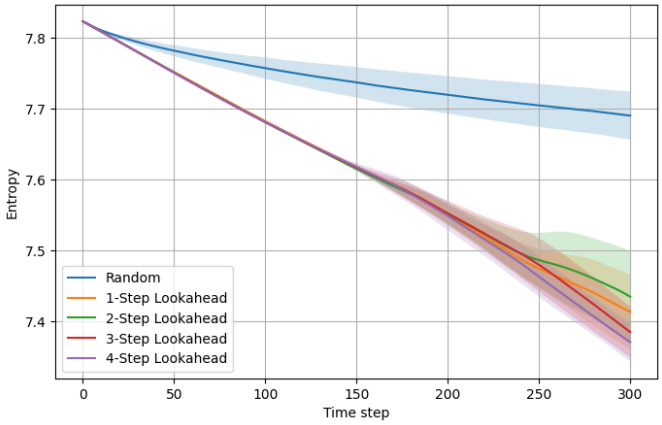


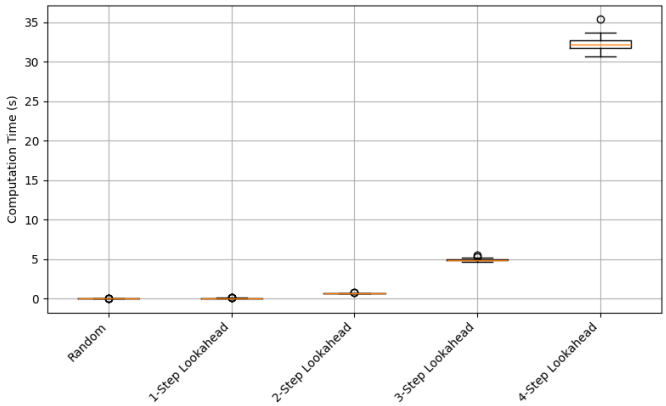
Fig. 7: Comparison of planner performance in target localization with detections enabled.

F. Effect of Planning Horizon

To understand the impact of horizon length on planning effectiveness, we vary the lookahead parameter $\mathcal{H} \in \{1, 2, 3, 4\}$ in a simplified setting with a single agent ($N = 1$) and no detections. This isolates the planning behavior, free of randomness, from the measurement model. Figure 8a shows the evolution of belief entropy over 100 randomized trials for different horizon lengths. We observe that longer horizons yield greater entropy reduction, especially in the later stages of the simulation, as the planner gains foresight into information-rich regions. However, this benefit is offset by a steep rise in computation time per step, shown in Figure 8b, reflecting the exponential complexity of such approaches. We expect these improvements to be more consequential in complex planning scenarios involving larger numbers of agents (N), but the doubly exponential time complexity makes such benchmarks difficult to conduct.



(a) Entropy trajectories for different planning horizons $\mathcal{H}$.



(b) Average computation time per planning step as a function of $\mathcal{H}$.

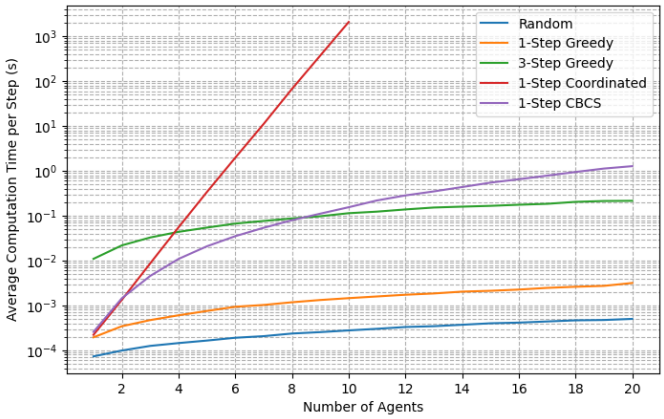
Fig. 8: Effect of increasing planning horizon on (a) entropy reduction and (b) computation time.

G. Computation Time Analysis

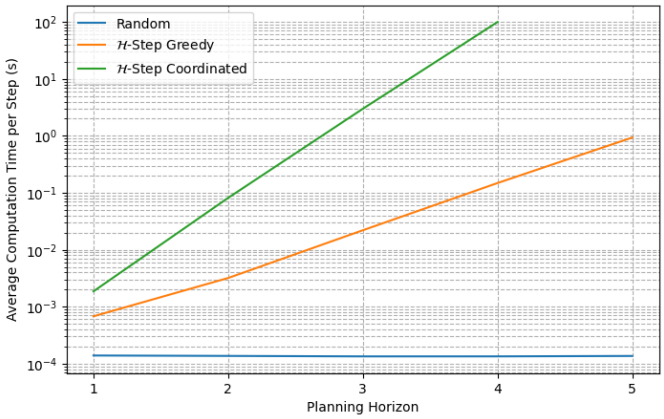
We now investigate the average computation time per planning step across different scenarios. Figure 9a demonstrates how increasing the number of agents N affects computational cost for a fixed planning horizon $\mathcal{H} = 1$. As expected, the 1-STEPGREEDY planner is computationally efficient, owing to its decoupled, per-agent planning. In contrast, the 1-STEPCOORDINATED planner incurs significantly higher costs because it must evaluate joint actions across all agents.

CBCS, while more computationally intensive than greedy planning, demonstrates significant computational advantages over the coordinated approach. With just $N = 10$ agents,

CBCS achieves a **13,417x** speedup over coordinated planning. Since CBCS decomposes the joint planning problem into a series of single-agent optimizations (potentially exponential in number, but individually tractable), the total planning time remains practical even for large teams. For instance, with $N = 20$ agents, CBCS requires approximately 1.27 seconds per planning step in a 50x50 grid world using an unclipped detection model. In scenarios where the sensor model is finite (e.g., disk-based models) or the state space is larger, we expect fewer conflicts requiring replanning, further reducing computation times.



(a) Computation time vs. number of agents N with fixed planning horizon $\mathcal{H} = 1$.



(b) Computation time vs. planning horizon $\mathcal{H}$ with $N = 2$ agents.

Fig. 9: Average computation time per planning step for various algorithmic configurations.

Subplot 9b explores the impact of increasing the planning horizon $\mathcal{H}$ while fixing the number of agents at $N = 2$. As expected, both greedy and coordinated planners show exponential growth in computation time with increasing $\mathcal{H}$, consistent with the theoretical analysis in Section VI-A. Future work will explore a receding-horizon implementation of CBCS, in which constraints are indexed by both agent and time (i.e., $c = (a_i, u, h)$), enabling planning instances to be decoupled not only across agents but also across future time steps.

VIII. Conclusion

This paper investigates cooperative multi-agent planning strategies for target localization under noisy measurements. We compare several planners—ranging from random baselines to coordinated and multi-step approaches—using two objective functions: mutual information maximization and detection probability maximization. Through extensive simulations, we evaluated these methods in both entropy minimization and target detection objectives.

Our results demonstrate that mutual information-based planners are more effective in reducing uncertainty when detections are disabled, while detection-based objectives achieve similar performance in scenarios with an active measurement model. Additionally, we introduced a novel hierarchical strategy, Conflict-Based Coordinated Search (CBCS), which achieves performance close to fully coordinated planning at a fraction of the computational cost. CBCS showed promising results in balancing solution optimality and scalability, making it well-suited for real-time multi-agent decision-making.

Several promising directions remain for future investigation. First, extending CBCS to receding-horizon planning could yield further improvements in coordination quality, but doing so requires careful algorithmic design to manage the combinatorial explosion in planning complexity. Second, theoretical analysis of CBCS is warranted. In particular, understanding the structural properties of global objective functions—and how individual agent choices influence them—could clarify the conditions under which CBCS can provide optimality guarantees.

References

[1] Gabriel M. Hoffmann and Claire J. Tomlin. Mobile sensor network control using mutual information methods and particle filters. *IEEE Transactions on Automatic Control*, 55(1):32–47, Jan 2010.

[2] Allison Ryan and J. Karl Hedrick. Particle filter based information-theoretic active sensing. *Robotics and Autonomous Systems*, 58(5):574–584, 2010.

[3] Frédéric Bourgault, Tomonari Furukawa, and Hugh F. Durrant-Whyte. *Optimal Search for a Lost Target in a Bayesian World*, pages 209–222. Springer Berlin Heidelberg, Berlin, Heidelberg, 2006.

[4] Ke Zhou and Stergios I. Roumeliotis. Optimal motion strategies for range-only constrained multisensor target tracking. *IEEE Transactions on Robotics*, 24(5):1168–1185, Oct 2008.

[5] T.H. Chung, J.W. Burdick, and R.M. Murray. A decentralized motion coordination strategy for dynamic target tracking. In *Proceedings 2006 IEEE International Conference on Robotics and Automation, 2006. ICRA 2006.*, pages 2416–2422, May 2006.

[6] Jingjin Yu and Steven LaValle. Structure and intractability of optimal multi-robot path planning on graphs. *Proceedings of the AAAI Conference on Artificial Intelligence*, 27(1):1443–1449, Jun. 2013.

[7] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015.